

Problem List

1704. Determine if String Halves Are Alike

EasyTopicsCompaniesHint

You are given a string `s` of even length. Split this string into two halves of equal lengths, and let `a` be the first half and `b` be the second half.

Two strings are **alike** if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'). Notice that `s` contains uppercase and lowercase letters.

Return `true` if `a` and `b` are **alike**. Otherwise, return `false`.

Example 1:

Input: `s = "book"`
Output: `true`
Explanation: `a = "bo"` and `b = "ok"`. `a` has 1 vowel and `b` has 1 vowel. Therefore, they are alike.

Example 2:

Input: `s = "textbook"`
Output: `false`
Explanation: `a = "text"` and `b = "book"`. `a` has 1 vowel whereas `b` has 2. Therefore, they are not alike. Notice that the vowel `o` is counted twice.

Constraints:

- `2 <= s.length <= 1000`
- `s.length` is even.
- `s` consists of uppercase and lowercase letters.

Code

Java

```
1 class Solution {
2     public boolean halvesAreAlike(String s) {
3         int mid = s.length() / 2;
4         int countA = 0;
5         int countB = 0;
6
7         for (int i = 0; i < mid; i++) {
8             if (isVowel(s.charAt(i))) {
9                 countA++;
10            }
11        }
12
13        for (int i = mid; i < s.length(); i++) {
14            if (isVowel(s.charAt(i))) {
15                countB++;
16            }
17        }
18
19        return countA == countB;
20    }
21
22    private boolean isVowel(char c) {
23        return "aeiouAEIOU".indexOf(c) != -1;
24    }
25 }
```

Test Result

Accepted

Runtime: 0 ms

Case 1Case 2

Input

s = "book"

Lapindromes

Lapindrome is defined as a string which when split in the middle, gives two halves having the same characters and same frequency of each character. If there are odd number of characters in the string, we ignore the middle character and check for lapindrome. For example **gaga** is a lapindrome, since the two halves **ga** and **ga** have the same characters with same frequency. Also, **abccab**, **rotor** and **xyzyx** are a few examples of lapindromes. Note that **abbaab** is NOT a lapindrome. The two halves contain the same characters but their frequencies do not match.

Your task is simple. Given a string, you need to tell if it is a lapindrome.

Input:

First line of input contains a single integer **T**, the number of test cases.
Each test is a single line containing a string **S** composed of only lowercase English alphabet.

Output:

For each test case, output on a separate line: "YES" if the string is a lapindrome and "NO" if it is not.

Constraints:

- $1 \leq T \leq 100$
- $2 \leq |S| \leq 1000$, where $|S|$ denotes the length of **S**

Sample 1:

Input	Output
6	YES
gaga	NO
abcde	YES
rotor	YES
xyzyx	NO
abbaab	NO

```
Java
19 // First half
20 for (int i = 0; i < mid; i++) {
21     freqLeft[s.charAt(i) - 'a']++;
22 }
23
24 // Second half
25 // For odd length, skip the middle character
26 int start = (n % 2 == 0) ? mid : mid + 1;
27
28 for (int i = start; i < n; i++) {
29     freqRight[s.charAt(i) - 'a']++;
30 }
31
32 // Compare frequencies
33 boolean isLapindrome = true;
34
35 for (int i = 0; i < 26; i++) {
36     if (freqLeft[i] != freqRight[i]) {
37         isLapindrome = false;
38         break;
39     }
40 }
```

Test against Custom Input

```
6
gaga
abcde
rotor
```

Status: Successfully executed

Time: 0.1000 secs

Memory: 42.144 Mb

Sample Input

```
6
gaga
```

Visualize Code

Run

Submit

Sample Input 0

Line: 81 Col: 1

Submit Code

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Download

1	5 6 7
2	3 6 10

Problem List<>

Submit

0

Premium

DescriptionEditorialSolutionsSubmissions

217. Contains Duplicate

EasyTopicsCompanies

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: `true`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

14.2K466

200 Online

Testcase

Code

JavaAuto

```
1 import java.util.HashSet;
2
3 class Solution {
4     public boolean containsDuplicate(int[] nums) {
5         HashSet<Integer> set = new HashSet<>();
6
7         for (int num : nums) {
8             if (set.contains(num)) {
9                 return true;
10            }
11            set.add(num);
12        }
13
14        return false;
15    }
16 }
17
```

Saved

Ln 14, Col 1

Test Result

Output

true

Expected

true

Contribute a testcase

Problem

Submissions

Leaderboard

Discussions

Editorial

Given a time in 12-hour *AM/PM* format, convert it to military (24-hour) time.

Notes: - 12:00:00AM on a 12-hour clock is 00:00:00 on a 24-hour clock.
- 12:00:00PM on a 12-hour clock is 12:00:00 on a 24-hour clock.

Example

- s = '12:01:00PM'
Return '12:01:00';
- s = '12:01:00AM'
Return '00:01:00'.

Function Description

Complete the *timeConversion* function with the following parameter(s):

- string s: a time in 12 hour format

Returns

- string: the time in 24 hour format

Input Format

A single string s that represents a time in 12-hour clock format (i.e.: hh:mm:ssAM or hh:mm:ssPM).

Constraints

- All input times are valid

Sample Input 0

07:05:45PM

Sample Output 0

19:05:45

```
18 // The function is expected to return a string.
19 // The function accepts STRING s as parameter.
20 //
21
22 public static String timeConversion(String s) {
23     String period = s.substring(8, 10);
24     int hour = Integer.parseInt(s.substring(0, 2));
25
26     if (period.equals("AM")) {
27         if (hour == 12) {
28             hour = 0;
29         }
30     } else {
31         if (hour != 12) {
32             hour += 12;
33         }
34     }
35
36     return String.format("%02d", hour) + s.substring(2, 8);
37 }
38
39 }
40
41 public class Solution {
42     public static void main(String[] args) throws IOException {
43         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
44         BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
45     }
46 }
```

Line: 36 Col: 60

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

07:05:45PM

Problem List

283. Move Zeroes

Description

Editorial

Solutions

Submissions

283. Move Zeroes

EasyTopicsCompassHint

Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`
Output: `[1,3,12,0,0]`

Example 2:

Input: `nums = [0]`
Output: `[0]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Could you minimize the total number of operations done?



Seen this question in a real interview before? 1/5

Yes

No

Accepted 536,985 of 538

Acceptance Rate 64.3%

19.8K

446

182 Online

Code

JavaAuto

```
1 class Solution {
2     public void moveZeroes(int[] nums) {
3         int i, j, t;
4         j = 0;
5
6         for (i = 0; i < nums.length; i++) {
7             if (nums[i] != 0) {
8                 t = nums[i];
9                 nums[i] = nums[j];
10                nums[j] = t;
11            }
12            j++;
13        }
14    }
15 }
16 }
```

SavedIn 16 C++2

Test Result

AcceptedRuntime: 0 ms

Case 1

Case 2

Input

nums = [0,1,0,3,12]

Output

Testcase

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerRank

Prepare Algorithms Warmup Diagonal Difference

Exit Full Screen View

Given a square matrix, calculate the absolute difference between the sums of its diagonals.

For example, the square matrix `arr` is shown below:

```
1 2 3
4 5 6
9 8 9
```

- The left-to-right diagonal = $1 + 5 + 9 = 15$.
- The right-to-left diagonal = $3 + 5 + 9 = 17$.

Their absolute difference is $|15 - 17| = 2$.

Function description

Complete the `diagonalDifference` function with the following parameter:

- `int arr[n][m]`: a 2-D array of integers

Return

- `int`: the absolute difference in sums along the diagonals

Input Format

The first line contains a single integer, `n`, the number of rows and columns in the square matrix `arr`.

Each of the next `n` lines describes a row, `arr[i]`, and consists of `n` space-separated integers `arr[i][j]`.

Constraints

- $-100 \leq arr[i][j] \leq 100$

Sample Input

STDIN	Function
3	<code>arr</code> [[]] sizes <code>n = 3</code> , <code>m = 3</code>
11 2 4	<code>arr = [[11, 2, 4], [4, 5, 6], [10, 8, -12]]</code>
4 5 6	
10 8 -12	

Sample Output

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

```
class Result {
    /*
     * Complete the 'diagonalDifference' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts 2D_INTEGER_ARRAY arr as parameter.
     */

    public static int diagonalDifference(List<List<Integer>> arr) {
        int n = arr.size();

        int primary = 0;
        int secondary = 0;

        for (int i = 0; i < n; i++) {
            // Left-to-right diagonal
            primary += arr.get(i).get(i);

            // Right-to-left diagonal
            secondary += arr.get(i).get(n - 1 - i);
        }

        return Math.abs(primary - secondary);
    }
}
```

Line: 36 Col: 42

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

3

Download

Problem List

867. Transpose Matrix

EasyTopicsCompaniesHint

Given a 2D integer array `matrix`, return the **transpose** of `matrix`.

The **transpose** of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

2	4	-1
-10	5	11
18	-7	6

→

2	-10	18
4	5	-7
-1	11	6

Example 1:

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output: `[[1,4,7],[2,5,8],[3,6,9]]`

Example 2:

Input: `matrix = [[1,2,3],[4,5,6]]`
Output: `[[1,4],[2,5],[3,6]]`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 3000`
- `1 <= m * n <= 105`

4.2K13317 Online

Code

JavaAuto

```
1 class Solution {
2     public int[][] transpose(int[][] matrix) {
3         int m = matrix.length;
4         int n = matrix[0].length;
5
6         int[][] result = new int[n][m];
7
8         for (int i = 0; i < m; i++) {
9             for (int j = 0; j < n; j++) {
10                 result[j][i] = matrix[i][j];
11             }
12         }
13
14         return result;
15     }
16 }
```

SavedUn 18, Col 2

Test Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

matrix =

[[1,2,3],[4,5,6],[7,8,9]]

Testcase

Problem List<>🔍

🏠📄📁📤

🔧🔍🔄🔁🔒👤🌟Premium

DescriptionEditorialSolutionsSubmissions

1314. Matrix Block Sum

Medium🔖Topics🏢Companies🔖Hint

Given a $m \times n$ matrix `mat` and an integer `k`, return a matrix `answer` where each `answer[i][j]` is the sum of all elements `mat[r][c]` for:

- $i - k \leq r \leq i + k$,
- $j - k \leq c \leq j + k$, and
- (r, c) is a valid position in the matrix.

Example 1:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 1`
Output: `[[12,21,16],[27,45,33],[24,39,28]]`

Example 2:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 2`
Output: `[[45,45,45],[45,45,45],[45,45,45]]`

Constraints:

- $m == mat.length$
- $n == mat[i].length$
- $1 \leq m, n, k \leq 100$
- $1 \leq mat[i][j] \leq 100$

Discover more📱Android Apps📱Programming

👍2.5K🗨️51🌟🔖🔄🔁

19 Online

Code

Java📄Auto

```
1 class Solution {
2     public int[][] matrixBlockSum(int[][] mat, int k) {
3         int m = mat.length;
4         int n = mat[0].length;
5
6         int[][] answer = new int[m][n];
7
8         for (int i = 0; i < m; i++) {
9             for (int j = 0; j < n; j++) {
10
11                 int sum = 0;
12
13                 int rowStart = Math.max(0, i - k);
14                 int rowEnd = Math.min(m - 1, i + k);
15
16                 int colStart = Math.max(0, j - k);
17                 int colEnd = Math.min(n - 1, j + k);
18
19                 for (int r = rowStart; r <= rowEnd; r++) {
20                     for (int c = colStart; c <= colEnd; c++) {
21                         sum += mat[r][c];
22                     }
23                 }
24
25                 answer[i][j] = sum;
26             }
27         }
28     }
29 }
```

Ln 31, Col 2

Test Result

AcceptedRuntime: 0 ms

Case 1Case 2

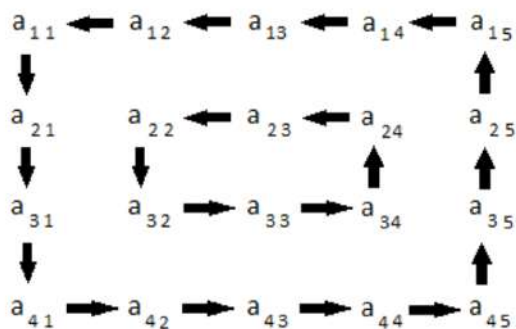
Input

mat =
[[1,2,3],[4,5,6],[7,8,9]]

Testcase

You are given a 2D matrix of dimension $m \times n$ and a positive integer r . You have to rotate the matrix r times and print the resultant matrix. Rotation should be in anti-clockwise direction.

Rotation of a 4×5 matrix is represented by the following figure. Note that in one rotation, you have to shift elements by one step only.



Matrix Rotation

It is guaranteed that the minimum of m and n will be even.

As an example rotate the Start matrix by 2:

Start	First	Second
-------	-------	--------

```

18 import static java.util.stream.Collectors.joining;
19 import static java.util.stream.Collectors.toList;
20
21 class Result {
22
23     /*
24      * Complete the 'matrixRotation' function below.
25      *
26      * The function accepts following parameters:
27      * 1. 2D_INTEGER_ARRAY matrix
28      * 2. INTEGER r
29      */
30
31     public static void matrixRotation(List<List<Integer>> matrix, int r) {
32         int m = matrix.size();
33         int n = matrix.get(0).size();
34
35         int layers = Math.min(m, n) / 2;
36
37         for (int layer = 0; layer < layers; layer++) {
38             List<Integer> values = new ArrayList<>();
39
40             int top = layer;
41             int bottom = m - 1 - layer;
42             int left = layer;
43             int right = n - 1 - layer;
44
45             // Rotate the current layer
46             // ... (implementation details) ...
47         }
48     }
49 }

```

Line: 99 Col: 5

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

1 4 4 1